

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский государственный университет
аэрокосмического приборостроения»

УЧЕБНАЯ ПРАКТИКА

(Первый курс. II семестр)

Методические указания

Автор: Гуков Сергей Юрьевич

Версия от 10.03.2026

Санкт-Петербург

2026

*Учебная практика длится весь семестр, защита проекта происходит на зачетной неделе. Однако сдать работу можно в течение всего семестра, не обязательно дожидаться официальной даты сдачи. На защиту необходимо приходить с **уже готовыми отчетом и программой**. Обратите внимание на дополнительную информацию, которая указана в описании задания в личном кабинете. Также можно обращаться со всеми возникшими вопросами и трудностями к преподавателю.*

Контакты для связи с преподавателем (Гуков Сергей Юрьевич):

- *Telegram: @bruimafia*
- *Email: sg_brui@mail.ru*

Контакты для связи с преподавателем (Савельева Дарья Дмитриевна):

- *Email: savel.dar.23r@gmail.com*

Учебная ознакомительная практика входит в состав обязательной части образовательной программы подготовки обучающихся по направлению подготовки/специальности 09.03.02 «Информационные системы и технологии». Организацию и проведение практики осуществляет кафедра №42.

Цели проведения учебной практики: закрепление и углубление теоретических знаний, полученных при изучении естественно-научных и профессиональных дисциплин; приобретение практических навыков и компетенций в сфере профессиональной деятельности; освоение студентами перспективных информационных технологий; приобретение опыта применения современной вычислительной техники для решения практических задач.

Задачи проведения учебной практики: умение самостоятельно или в составе научно-производственного коллектива решать конкретные профессиональные задачи; формирование и развитие у студентов профессионально значимых качеств, устойчивого интереса к профессиональной деятельности, потребности в самообразовании; изучение специальной литературы и другой научно-технической информации, достижений отечественной и зарубежной науки и техники в области информационных технологий и систем; сбор, обработка, анализ и систематизация научно-технической информации по теме (заданию); оформление результатов анализа информации по заданной теме и собственных исследований и разработок в виде отчета и технической документации.

Учебная ознакомительная практика обеспечивает формирование у обучающихся следующих компетенций:

- УК-6 «Способен управлять своим временем, выстраивать и реализовывать траекторию саморазвития на основе принципов образования в течение всей жизни»;
- ОПК-1 «Способен применять естественнонаучные и общетехнические знания, методы математического анализа и моделирования, теоретического и экспериментального исследования в профессиональной деятельности»;
- ОПК-2 «Способен понимать принципы работы современных информационных технологий и программных средств, в том числе отечественного производства, и использовать их при решении задач профессиональной деятельности»;
- ОПК-3 «Способен решать стандартные задачи профессиональной деятельности на основе информационной и библиографической культуры с применением информационно-коммуникационных технологий и с учетом основных требований информационной безопасности»;
- ОПК-4 «Способен участвовать в разработке технической документации, связанной с профессиональной деятельностью с использованием стандартов, норм и правил»;
- ОПК-6 «Способен разрабатывать алгоритмы и программы, пригодные для практического применения в области информационных систем и технологий»;
- ПК-3 «Способен разрабатывать программное обеспечение, выполнять интеграцию программных модулей и компонентов».

Содержание практики охватывает круг вопросов, связанных с разработкой программного обеспечения и написания для него технической документации.

Аттестация по практике осуществляется путем защиты отчетов, составляемых обучающимися по итогам практики. Форма аттестации по практике – дифференцированный зачет.

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

В рамках учебной практики необходимо закрепить и расширить полученные знания по программированию, разработав полноценное игровое приложение с использованием игрового

движка или специализированной библиотеки, а также оформив техническую документацию к нему.

Необходимо реализовать игру с применением одного из игровых движков или сторонних игровых библиотек. Язык программирования не ограничен – выбирается исходя из возможностей выбранного инструмента. Разработка «с нуля» без использования каких-либо движков или библиотек не допускается. Исключение возможно лишь в случае реализации консольной игры с нетривиальной алгоритмической сложностью – такой вариант необходимо отдельно согласовать с преподавателем.

Игра должна отвечать следующим критериям:

- наличие как минимум двух режимов: игра двух игроков и/или игра против компьютера (если игровой жанр это предполагает);
- несколько уровней сложности (где это применимо);
- достаточная функциональность и завершённость – игра должна представлять собой цельный, играбельный продукт.

Далее перечислены примеры некоторых движков и библиотек, однако не запрещается использовать и другие.

Игровые движки:

- Godot (GDScript, C#, C++) – универсальный движок с открытым исходным кодом, хорошо подходит для начинающих
- Unity (C#) – один из самых популярных движков, огромная документация и сообщество
- Unreal Engine (C++, Blueprints) – мощный движок с визуальным скриптингом, подходит для амбициозных проектов
- Pygame (Python) – простой старт для 2D-игр, минимальный порог вхождения
- LÖVE / Love2D (Lua) – лёгкий движок для 2D, очень быстрый старт
- GDevelop (без кода / JavaScript) – визуальная разработка, не требует глубоких знаний программирования
- Defold (Lua) – компактный движок от King, хорошо документирован
- MonoGame (C#) – фреймворк для 2D/3D, наследник XNA, популярен среди инди-разработчиков
- Phaser (JavaScript/TypeScript) – движок для браузерных 2D-игр

- Cocos2d-x (C++, Lua, JavaScript) – кроссплатформенный движок для 2D
- HaxeFlixel (Haxe) – 2D-движок с простым синтаксисом, похожим на ActionScript
- Heaps.io (Haxe) – используется в реальных коммерческих играх (Dead Cells)
- Bevy (Rust) – современный движок на Rust с ECS-архитектурой, для тех, кто хочет попробовать что-то необычное
- LibGDX (Java/Kotlin) – кроссплатформенный фреймворк, популярен среди Java-разработчиков
- Axys / Cocos Creator (JavaScript/TypeScript) – визуальная среда для 2D/3D игр

Библиотеки:

- SFML (C++) – графика, звук, ввод; хорошая альтернатива для тех, кто хочет писать на C++
- SDL2 (C/C++) – низкоуровневая мультимедийная библиотека, широко используется в индустрии
- Raylib (C/C++) – минималистичная и дружелюбная библиотека, отлично подходит для учебных проектов
- Allegro (C/C++) – классическая игровая библиотека, простая в освоении
- OpenFL (Haxe) – реализация Flash API, подходит для 2D-игр
- Arcade (Python) – более современная альтернатива Pygame с удобным API
- Ren'Py (Python) – специализирован для визуальных новелл и текстовых квестов
- Pyxel (Python) – ретро-игровой движок в стиле fantasy-консоли (ограниченная палитра, пиксель-арт)
- ncurses / PDCurses (C/C++) – терминальная графика; допускается только при особой сложности проекта, по согласованию
- Lanterna (Java) – терминальная библиотека для консольных интерфейсов на Java
- BearLibTerminal (C/C++, биндинги на Python, C# и др.) – псевдографика для roguelike-игр
- libtcod (C/C++, Python) – классическая библиотека для roguelike, включает FOV, pathfinding и другое
- Three.js (JavaScript) – 3D в браузере, можно делать простые 3D-игры без движка
- PixiJS (JavaScript) – быстрый 2D-рендерер для браузера на WebGL
- Godot GDNative / GDExtension – позволяет подключать нативные модули на C++ к Godot

Готовый проект необходимо разместить на GitHub или аналогичной платформе контроля версий с оформленным файлом «README.md», содержащим описание игры, инструкцию по запуску и использованные технологии.

На защите необходимо представить исходный код проекта и оформленный отчёт, а также быть готовым ответить на теоретические вопросы по коду и применённым инструментам. При несогласии с предлагаемой оценкой можно выполнить небольшое практическое задание непосредственно на защите для получения дополнительных баллов.

ЧАСТЬ 1. РЕАЛИЗАЦИЯ ЗАДАНИЯ

Что такое GitHub и как с ним работать?

Git — распределённая система контроля версий, которая даёт возможность разработчикам отслеживать изменения в файлах и работать над одним проектом совместно с коллегами.

Чтобы лучше понимать, что такое Git и как он работает, нужно знать, что такое система контроля версий. Системы контроля версий (СКВ, VCS, Version Control Systems) позволяют разработчикам сохранять все изменения, внесённые в код. При возникновении проблем разработчики могут просто откатить код до рабочего состояния и не тратить часы на поиски ошибок. СКВ также позволяют нескольким разработчикам работать над одним проектом и сохранять внесённые изменения независимо друг от друга. При этом каждый участник команды видит, над чем работают коллеги.

GitHub — наиболее популярный сервис онлайн-хостинга репозиториях, обладающий всеми функциями распределённого контроля версий и функциональностью управления исходным кодом — всё, что поддерживает Git и даже больше. Также GitHub может похвастаться контролем доступа, багтрекингом, управлением задачами и вики для каждого проекта. Кроме GitHub есть другие сервисы, которые используют Git, — например, Bitbucket и GitLab. Вы можете разместить свой проект на любом из них.

Рассмотрим работу с репозиториями на примере GitHub. Для работы с GitHub необходимо зайти на сайт <https://github.com/> и создать аккаунт. В целом есть несколько вариантов, как залить/управлять свой репозиторий (проект) на сайт. Рассмотрим один из них.

В программе Visual Studio есть вкладка «Git», в которой необходимо «Создать репозиторий» и ввести данные от своего аккаунта (рисунок 11). Далее, выбрав имя репозитория, отправить его на GitHub.

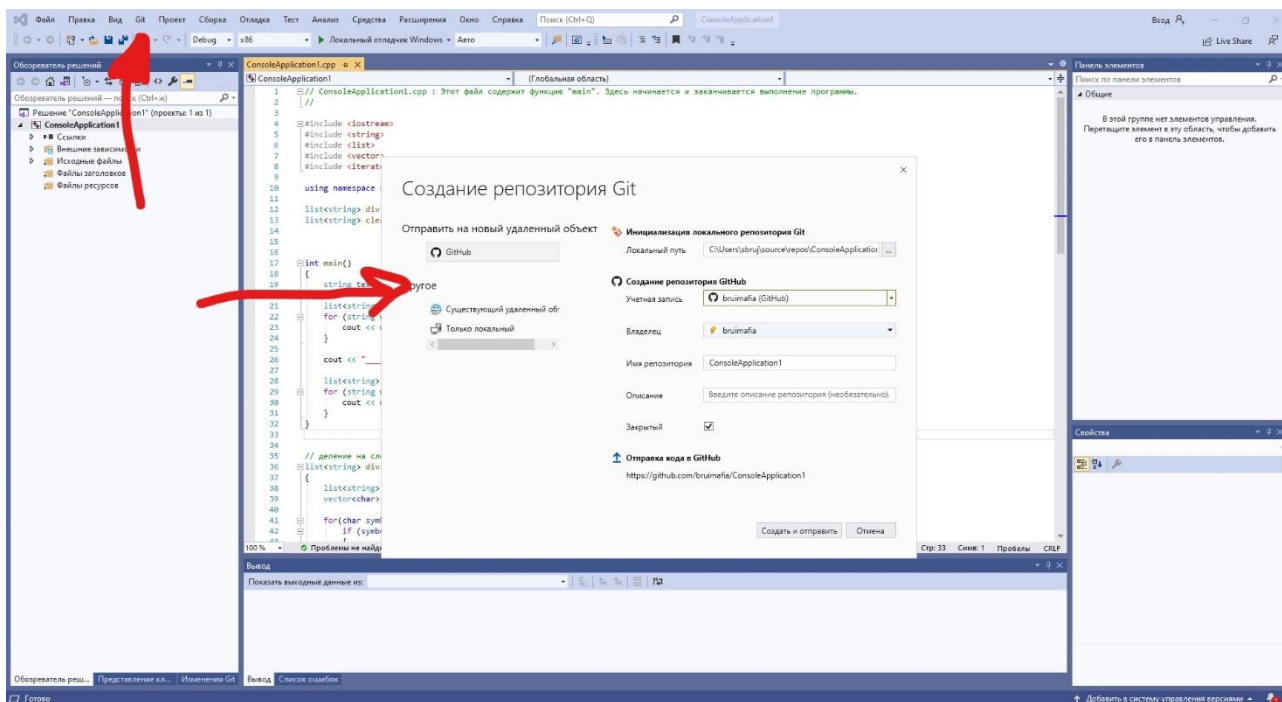


Рисунок 11 – Создание репозитория

Теперь во вкладке «Git» будет доступен примерно следующий функционал (рисунок 12). А в нижней вкладке «Изменения Git» появится возможность отправлять коммиты.

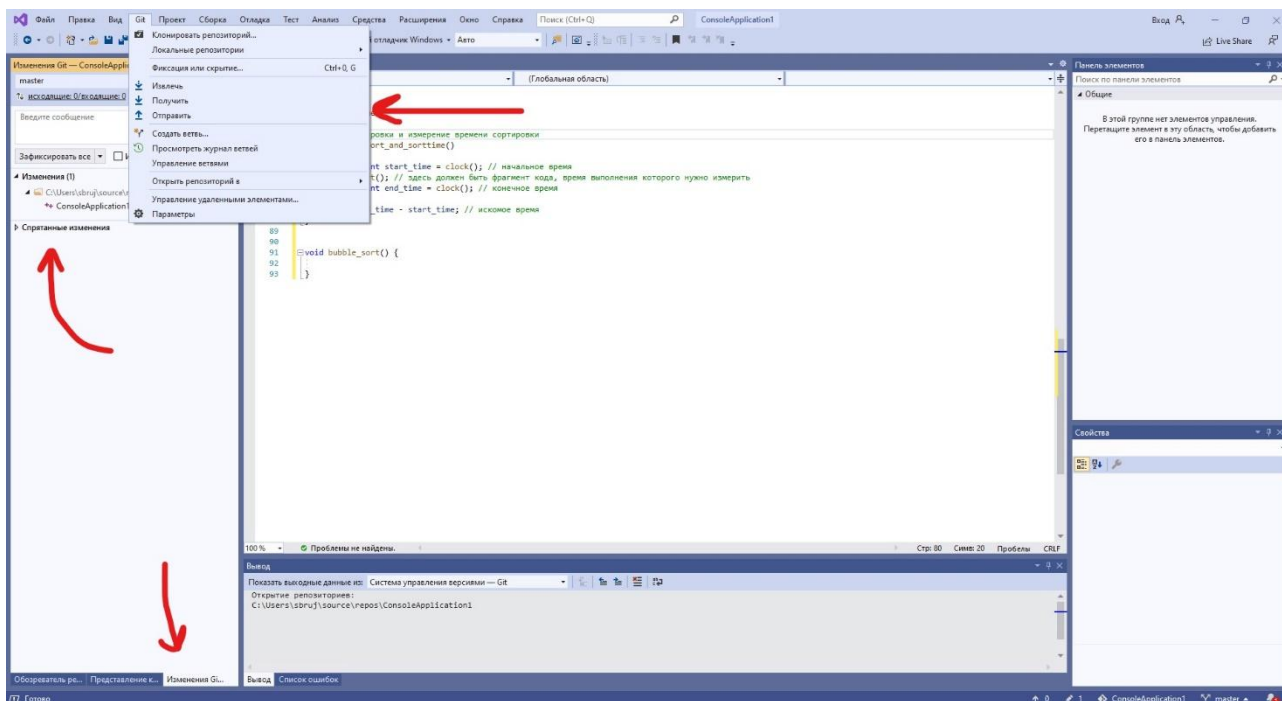


Рисунок 12 – Управление репозиторием

Git-коммит – это операция, которая берет все подготовленные изменения и отправляет их в репозиторий как единое целое. Каждый коммит в git репозитории хранит снимок всех файлов в репозитории. Почти как огромная копия, только эффективнее. Git пытается быть лёгким и быстрым, так что он не просто слепо копирует весь проект каждый раз, а ужимает коммит в набор изменений или «дельту» между текущей версией и предыдущей. Это позволяет занимать меньше места. Также Git хранит всю историю о том, когда какой коммит был сделан и кем. Это очень важно, так как можно посмотреть, из-за чего и из-за кого сломался весь код и «надавать ему по башке».

Возможность отправить коммит появляется после любого мельчайшего изменения кода. Однако делать это желательно после каждого идеологического/алгоритмического изменения кода программы. Перед нажатием «Зафиксировать все и отправить» следует ввести имя/описание коммита, отражающее причину изменения либо нововведения кода (рисунок 13).

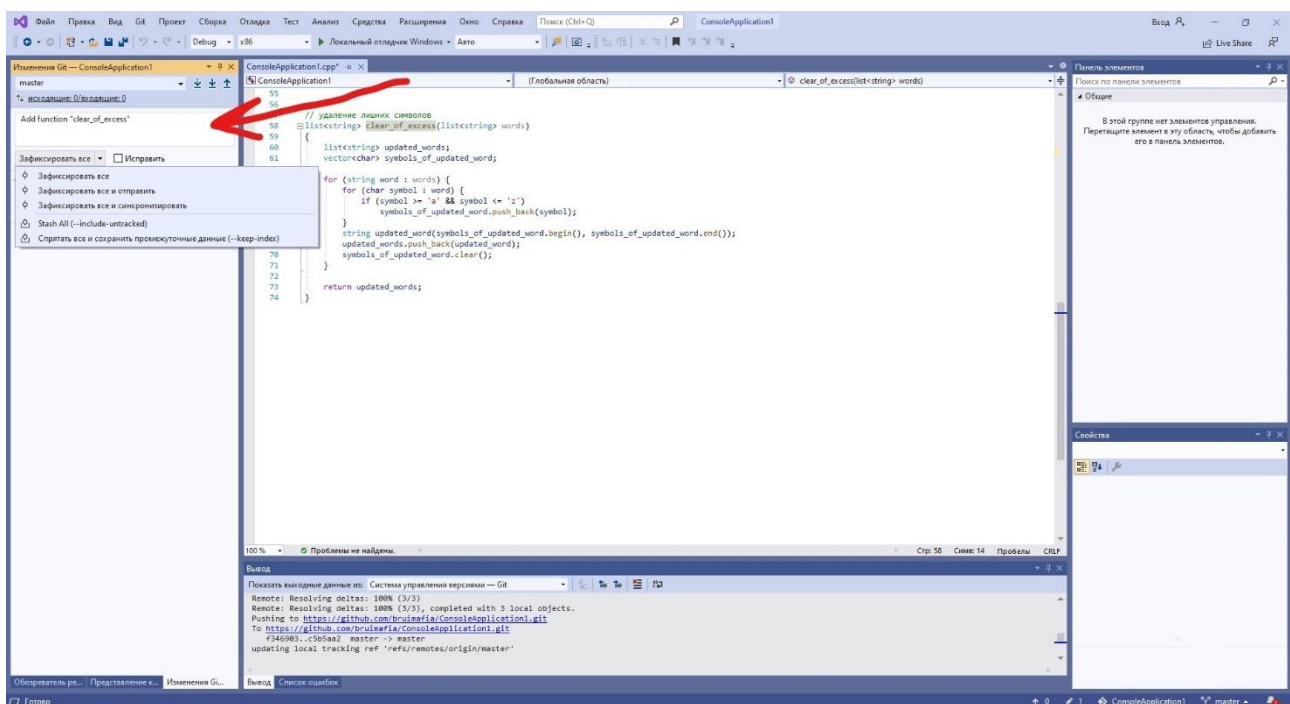


Рисунок 13 – Создание коммита

Для описания проектов на GitHub используется файл «README.md», который пишется на языке разметки markdown. «README.md» («прочти меня») – это первый файл, который нужно читать, получив доступ к проекту на GitHub или любой Git-хостинговой площадке. Этот файл в первую очередь и предлагается вниманию пользователя, когда он открывает здесь репозиторий того или иного проекта. Такой файл содержит кучу полезной информации, так что его вполне можно рассматривать как справочное руководство по проекту. Файл

«README.md» находится в корневой папке репозитория и автоматически отображается в каталоге проекта на GitHub. Примеры файлов «README.md» приведены на рисунках 14-16.

При составлении описания проекта, то есть файла «README.md», обычно придерживаются такого плана:

- название проекта;
- описание проекта (с использованием слов и изображений);
- демо (изображения, ссылки на видео, интерактивные демо-ссылки);
- технологии/фишки, использованные в проекте;
- что-то характерное для проекта (проблемы, с которыми пришлось столкнуться, уникальные составляющие проекта);
- техническое описание проекта (установка, настройка, как помочь проекту).

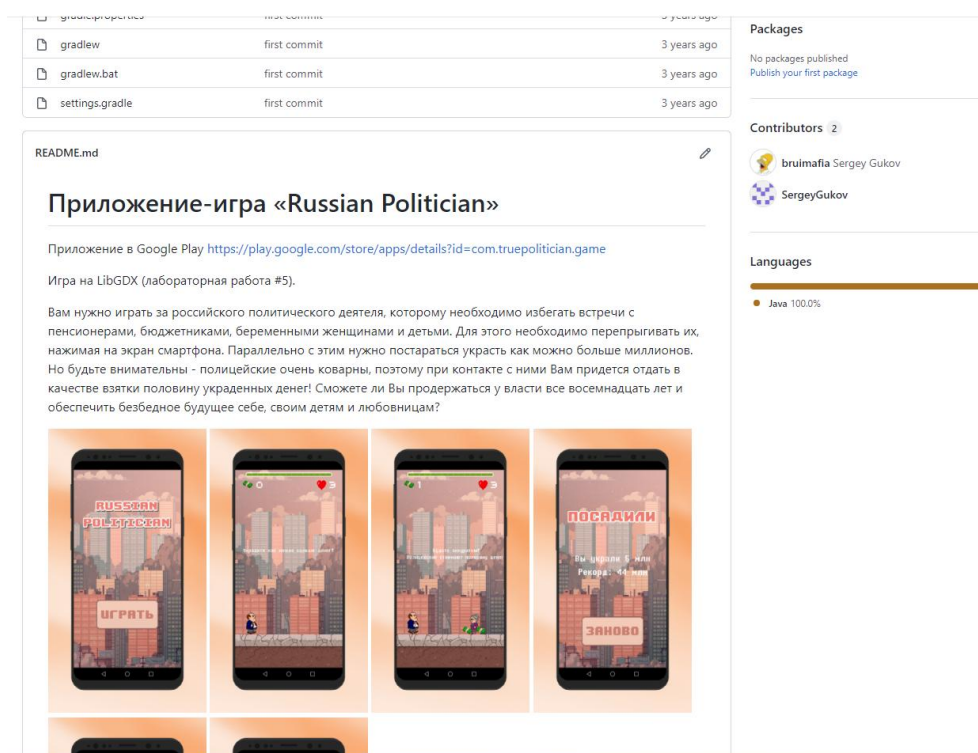


Рисунок 14 – Пример файла «README.md»

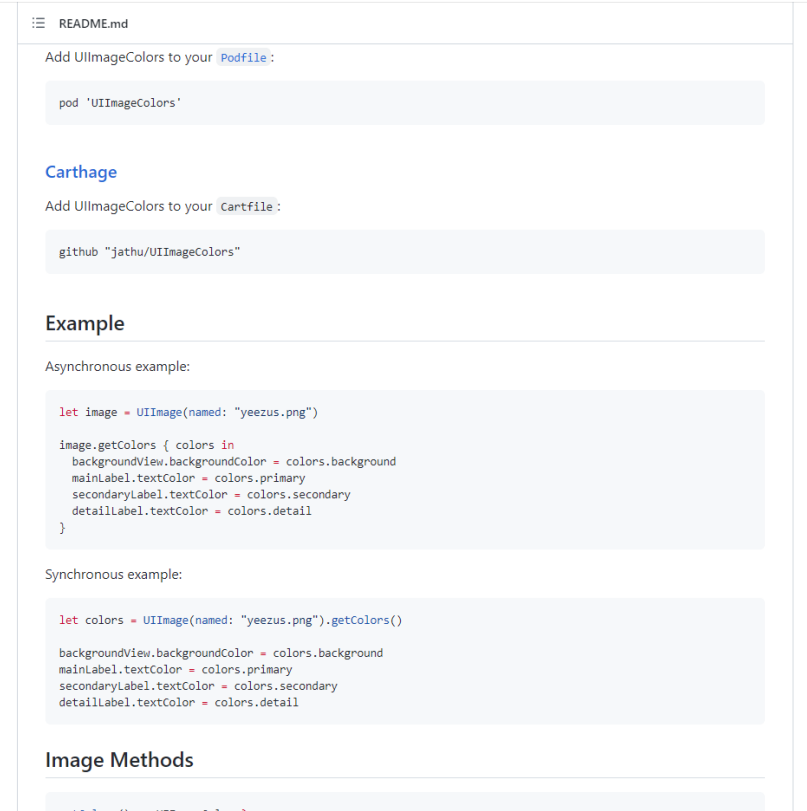


Рисунок 15 – Пример файла «README.md»

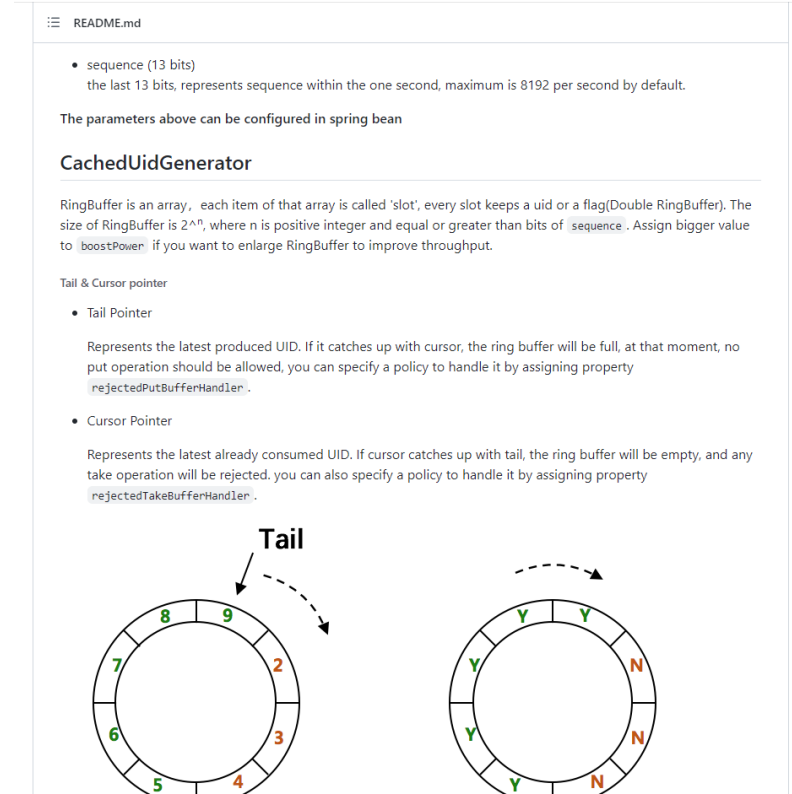


Рисунок 16 – Пример файла «README.md»

В общем, настоятельно рекомендуется познакомиться/поиграться с GitHub или другими подобными сервисами, так как практически в любой компании при разработке проекта используются системы контроля версий.

ЧАСТЬ 2. НАПИСАНИЕ ДОКУМЕНТАЦИИ

Рано или поздно разработчик сталкивается с ситуацией, когда не может понять написанный код программы. Такое случается и когда он приходит в новый для себя проект, и когда коллеги проверяют его код, и даже, когда он смотрит результат своей же работы. Чтобы избежать подобных проблем и упростить поддержку проекта, необходимо писать и читать документацию.

Для разработчика есть два основных вида документации:

– *Пользовательская документация.* Это руководство по эксплуатации программ. Обычно оно нужно для сложных профессиональных инструментов и является избыточным для небольших проектов. Если пользователи не могут сами разобраться в приложении заказа такси, то скорее всего проблемы в самом интерфейсе и лучше его доработать и упростить.

– *Техническая документация.* Это пояснения для программистов, которые будут использовать или дорабатывать существующий код. Они помогут быстро вникнуть в проект и начать работать. Или же продолжить самому разработчику писать программу после долгого перерыва.

Основные правила написания технической документации:

– *Документация нужна не всегда.* Если программа выглядит как небольшой скрипт, не стоит тратить время на написание пояснений.

– *Документация нужна не везде.* Если код написан хорошо, то по названиям уже будет понятно, что это такое и зачем оно используется. Исключение можно сделать, если речь идет об API или фреймворке, которыми пользуются многие разработчики: они не всегда видят исходный код, но могут использовать классы и методы.

– *Документация должна быть точной.* Очень важно уметь ясно выражать свои мысли. Нужно предельно точно описывать, что делает тот или иной фрагмент кода.

– *Документация должна быть сухой.* Нужно писать надо максимально сухо и по делу. Избегать стихов, метафор, аналогий, шуток и т.п.

– В документации не должно быть старого кода. Старайтесь не хранить в коде старые методы и операторы, даже если они закомментированы. Если что-то не используется в текущий момент, то от которого нужно избавиться. Если есть сомнения пригодится ли еще этот код, его лучше сохранить в системе контроля версий.

Ниже на рисунках 17-20 представлен один из вариантов хорошего описания функций (методов) и переменных (полей). Более подробную информацию об этих примерах можно посмотреть по следующим ссылкам:

- <https://yandex.ru/dev/mobile-ads/doc/android/ref/com/yandex/mobile/ads/banner/AdSize.html>
- <https://yandex.ru/dev/mobile-ads/doc/android/ref/com/yandex/mobile/ads/nativeads/Rating.html>
- <https://yandex.ru/dev/mobile-ads/doc/android/ref/com/yandex/mobile/ads/rewarded/RewardedAd.html>

Modifier and Type	Method and Description
static AdSize	<code>flexibleSize(int width, int height)</code> Задаёт размеры баннера с учетом размеров контейнера.
int	<code>getHeight()</code> Возвращает высоту баннера в dp (density-independent pixels).
int	<code>getHeight(android.content.Context context)</code> Возвращает высоту баннера в dp (density-independent pixels).
int	<code>getHeightInPixels(android.content.Context context)</code> Возвращает высоту баннера в пикселях.
int	<code>getWidth()</code> Возвращает ширину баннера в dp (density-independent pixels).
int	<code>getWidth(android.content.Context context)</code> Возвращает ширину баннера в dp (density-independent pixels).
int	<code>getWidthInPixels(android.content.Context context)</code> Возвращает ширину баннера в пикселях.
static AdSize	<code>stickySize(int width)</code> Задаёт ширину sticky баннера.

Рисунок 17 – Описание функций (слева – тип возвращаемого значения, справа – имя и назначение)

Modifier and Type	Method and Description
float	<code>getRating()</code> Возвращает значение рейтинга.
void	<code>setRating(float rating)</code> Задаёт значение рейтинга.

Рисунок 18 – Описание функций (слева – тип возвращаемого значения, справа – имя и назначение)

Modifier and Type	Method and Description
void	<code>destroy()</code> Удаляет объект класса <code>RewardedAd</code> и очищает все занятые ресурсы.
boolean	<code>isLoaded()</code> Возвращает результат загрузки рекламы.
void	<code>loadAd(AdRequest adRequest)</code> Начинает загрузку рекламы в фоновом режиме.
void	<code>setAdUnitId(java.lang.String adUnitId)</code> Задаёт уникальный идентификатор рекламного места.
void	<code>setRewardedAdEventListener(RewardedAdEventListener rewardedAdEventListener)</code> Задаёт объект класса <code>RewardedAdEventListener</code> для получения оповещений о событиях, происходящих во время жизненного цикла рекламы с вознаграждением.
void	<code>show()</code> Показывает рекламу, если она была загружена.

Рисунок 19 – Описание функций (слева – тип возвращаемого значения, справа – имя и назначение)

Modifier and Type	Field and Description
static <code>AdSize</code>	<code>BANNER_240x400</code> Баннер с размерами 240x400 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_300x250</code> Баннер с размерами 300x250 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_300x300</code> Баннер с размерами 300x300 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_320x100</code> Баннер с размерами 320x100 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_320x50</code> Баннер с размерами 320x50 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_400x240</code> Баннер с размерами 400x240 в dp (density-independent pixels).
static <code>AdSize</code>	<code>BANNER_728x90</code> Баннер с размерами 728x90 в dp (density-independent pixels).
static <code>AdSize</code>	<code>FULL_SCREEN</code> Баннер на весь размер экрана.

Рисунок 20 – Описание переменных (слева – тип, справа – имя и назначение)

Существуют различные варианты и стандарты по написанию документации. В случае выполнения задания по учебной практике можно использовать:

- блок-схему программы;
- краткие комментарии в коде (однострочные, многострочные);
- описание функций и переменных (как на примерах выше);
- описание алгоритмов программы;
- описание взаимодействия пользователя с программой.

ЧАСТЬ 3. ТРЕБОВАНИЯ К ОТЧЕТУ

Весь программный код должен быть разбит на отдельные функции/методы. Имена функций и переменных должны отражать их назначение. Основные части кода должны иметь комментарии. Желательно придерживаться главных пунктов конвенции по оформлению кода языка, на котором пишется проект.

Объем отчета не менее 17 страниц без учета приложения (в приложении весь исходный код).

Образцы титульного листа по практике и бланка индивидуального задания по практике размещены на сайте ГУАП в разделе Нормативная документация в строке «Документация для учебного процесса».

Отчет оформляется в соответствии с требованиями, описанными на сайте ГУАП в разделе Нормативная документация (URL: <https://guap.ru/standart/doc>).

В отчете по учебной практике должно быть следующее:

- титульный лист;
- индивидуальное задание;
- содержание;
- введение;
- тема и описание игры;
- пользовательская документация (описание взаимодействия пользователя с программой);
- техническая документация (блок-схема, комментарии в коде, описание алгоритмов, описание функций и переменных);

- тестирование программы;
- ссылка на короткое видео с демонстрацией механики игры;
- описание назначения и процесса загрузки проекта на Github (реализовать 2-4 коммита с описанием изменений), скриншоты файла документации «README.md» (в этом файле тоже указать ссылку на видео с демонстрацией механики игры);
- заключение;
- список использованных источников;
- приложение с исходным кодом.

СПИСОК ЛИТЕРАТУРЫ

1. Дополнительные материалы в личном кабинете
2. Конспекты лекций
3. Алгоритмы и структуры данных, Н. Вирт
4. Программирование. Принципы и практика с использованием C++, Страуструп Бьярне
5. Язык программирования C++. Лекции и упражнения, Прата Стивен
6. Язык программирования C++. Базовый курс, Липпман Стенли Б., Лажойе Жози
7. 90 рекомендаций по стилю написания программ на C++, <https://habr.com/ru/post/172091/> (дата обращения: 10.03.2026)
8. Гайд по оформлению кода на C++ от Стэнфордского университета, <https://tproger.ru/translations/stanford-cpp-style-guide/> (дата обращения: 10.03.2026)
9. Основы C++. Программирование для начинающих (канал «#SimpleCode»), <https://www.youtube.com/watch?v=kRcbYLK3OnQ&list=PLQOaTSbfxUtCrKs0nicOg2npJQYSPGO9r> (дата обращения: 10.03.2026)
10. C++ программирование / Уроки C++ (канал «Гоша Дударь»), https://www.youtube.com/watch?v=qSHP98i9mDU&list=PL0lO_mIqDDFXNfqIL9PHQM7Wg_kOtDZsW (дата обращения: 10.03.2026)
11. Краткое введение в Git и его использование с GitHub - GitHub - ashtanyuk/Git-intro: Краткое введение в Git и его использование с GitHub, <https://github.com/ashtanyuk/Git-intro> (дата обращения: 10.03.2026)
12. Godot Engine. Официальная документация [Электронный ресурс]. — Режим доступа: <https://docs.godotengine.org> (дата обращения: 10.03.2026).

13. Unity Technologies. Unity Documentation [Электронный ресурс]. — Режим доступа: <https://docs.unity3d.com> (дата обращения: 10.03.2026).
14. Epic Games. Unreal Engine Documentation [Электронный ресурс]. — Режим доступа: <https://dev.epicgames.com/documentation> (дата обращения: 10.03.2026).
15. Pygame. Pygame Documentation [Электронный ресурс]. — Режим доступа: <https://www.pygame.org/docs> (дата обращения: 10.03.2026).
16. LÖVE. Love2D Wiki [Электронный ресурс]. — Режим доступа: <https://love2d.org/wiki> (дата обращения: 10.03.2026).
17. GDevelop. GDevelop Documentation [Электронный ресурс]. — Режим доступа: <https://wiki.gdevelop.io> (дата обращения: 10.03.2026).
18. Defold. Defold Documentation [Электронный ресурс]. — Режим доступа: <https://defold.com/manuals> (дата обращения: 10.03.2026).
19. MonoGame. MonoGame Documentation [Электронный ресурс]. — Режим доступа: <https://docs.monogame.net> (дата обращения: 10.03.2026).
20. Phaser. Phaser 3 Documentation [Электронный ресурс]. — Режим доступа: <https://newdocs.phaser.io> (дата обращения: 10.03.2026).
21. Cocos2d-x. Cocos2d-x Programmer's Guide [Электронный ресурс]. — Режим доступа: <https://docs.cocos2d-x.org> (дата обращения: 10.03.2026).
22. HaxeFlixel. HaxeFlixel Documentation [Электронный ресурс]. — Режим доступа: <https://haxelexel.com/documentation> (дата обращения: 10.03.2026).
23. Heaps.io. Heaps Documentation [Электронный ресурс]. — Режим доступа: <https://heaps.io/documentation> (дата обращения: 10.03.2026).
24. Bevy. Bevy Engine Documentation [Электронный ресурс]. — Режим доступа: <https://bevyengine.org/learn> (дата обращения: 10.03.2026).
25. LibGDX. LibGDX Wiki [Электронный ресурс]. — Режим доступа: <https://libgdx.com/wiki> (дата обращения: 10.03.2026).
26. SFML. SFML Documentation [Электронный ресурс]. — Режим доступа: <https://www.sfml-dev.org/documentation> (дата обращения: 10.03.2026).
27. SDL. SDL2 Wiki [Электронный ресурс]. — Режим доступа: <https://wiki.libsdl.org> (дата обращения: 10.03.2026).
28. Raylib. Raylib Documentation [Электронный ресурс]. — Режим доступа: <https://www.raylib.com> (дата обращения: 10.03.2026).
29. Allegro. Allegro 5 Documentation [Электронный ресурс]. — Режим доступа: <https://liballeg.org/a5docs/trunk> (дата обращения: 10.03.2026).

30. Arcade. Python Arcade Documentation [Электронный ресурс]. — Режим доступа: <https://api.arcade.academy> (дата обращения: 10.03.2026).
31. Ren'Py. Ren'Py Documentation [Электронный ресурс]. — Режим доступа: <https://www.renpy.org/doc/html> (дата обращения: 10.03.2026).
32. Pyxel. Pyxel — Retro Game Engine for Python [Электронный ресурс] / К. Kitao. — Режим доступа: <https://github.com/kitao/pyxel> (дата обращения: 10.03.2026).
33. libtcod. libtcod Documentation [Электронный ресурс]. — Режим доступа: <https://python-tcod.readthedocs.io> (дата обращения: 10.03.2026).
34. BearLibTerminal. BearLibTerminal Documentation [Электронный ресурс]. — Режим доступа: <http://foo.wyrd.name/en:bearlibterminal> (дата обращения: 10.03.2026).
35. Three.js. Three.js Documentation [Электронный ресурс]. — Режим доступа: <https://threejs.org/docs> (дата обращения: 10.03.2026).
36. PixiJS. PixiJS Documentation [Электронный ресурс]. — Режим доступа: <https://pixijs.download/release/docs> (дата обращения: 10.03.2026).
37. Памятка начинающему разработчику компьютерных игр [Электронный ресурс] // Хабр. — 2019. — Режим доступа: <https://habr.com/ru/articles/470199/> (дата обращения: 10.03.2026).
38. Из Unity в Godot. Первое впечатление [Электронный ресурс] // Хабр. — 2021. — Режим доступа: <https://habr.com/ru/articles/571692/> (дата обращения: 10.03.2026).
39. Введение в программирование: заготовка игры-платформера на SDL в 300 строк C++ [Электронный ресурс] // Хабр. — 2021. — Режим доступа: <https://habr.com/ru/articles/578018/> (дата обращения: 10.03.2026).
40. Рефакторинг игры на SFML [Электронный ресурс] // Хабр. — 2019. — Режим доступа: <https://habr.com/ru/articles/480710/> (дата обращения: 10.03.2026).
41. 2D примитивы мультимедийной библиотеки SFML для разработки игр на C++ [Электронный ресурс] // Хабр. — 2022. — Режим доступа: <https://habr.com/ru/articles/702128/> (дата обращения: 10.03.2026).
42. Разработка игры на C++/SFML: Начало [Электронный ресурс] // Хабр. — 2024. — Режим доступа: <https://habr.com/ru/articles/800691/> (дата обращения: 10.03.2026).
43. Godot — это не новая Unity. Анатомия вызова API в Godot [Электронный ресурс] // Хабр. — 2023. — Режим доступа: <https://habr.com/ru/articles/763988/> (дата обращения: 10.03.2026).
44. Сделать 10 игр на Godot. От Pong до Portal. И вот итог [Электронный ресурс] // Хабр. — 2025. — Режим доступа: <https://habr.com/ru/articles/880904/> (дата обращения: 10.03.2026).

45. Выбор начинающего C++ gamedeveloper-а: SDL, SFML или что-то ещё? [Электронный ресурс] // Хабр Q&A. — Режим доступа: <https://qna.habr.com/q/290420> (дата обращения: 10.03.2026).
46. Какие игровые движки существуют для Python? [Электронный ресурс] // Хабр Q&A. — Режим доступа: <https://qna.habr.com/q/399492> (дата обращения: 10.03.2026).
47. Почему разработка игр на Python не умерла: Godot и другие библиотеки в геймдеве [Электронный ресурс] // tproger.ru. — 2024. — Режим доступа: <https://tproger.ru/articles/pochemu-razrabotka-igr-na-python-ne-umerla--godot-i-drugie-biblioteki-v-gejmdeve> (дата обращения: 10.03.2026).
48. Игры на Python: примеры и перспективы [Электронный ресурс] // GeekBrains. — 2024. — Режим доступа: <https://gb.ru/blog/igry-na-python/> (дата обращения: 10.03.2026).
49. Игровые движки — топ лучших движков для разработки игр [Электронный ресурс] // Timeweb Community. — 2022. — Режим доступа: <https://timeweb.com/ru/community/articles/10-luchshih-dvizhkov-dlya-sozdaniya-igr> (дата обращения: 10.03.2026).
50. Учебник по LibGDX [Электронный ресурс] // Gamedev Suffering (suvitruf.ru). — Режим доступа: <https://suvitruf.ru/libgdx/> (дата обращения: 10.03.2026).
51. Phaser vs PixiJS for making 2D games [Electronic resource] // DEV Community. — 2022. — Access mode: <https://dev.to/ritza/phaser-vs-pixijs-for-making-2d-games-2j8c> (дата обращения: 10.03.2026).
52. The Inside Scoop on LibGDX vs MonoGame [Electronic resource] // Aircada Blog. — Access mode: <https://aircada.com/blog/libgdx-vs-monogame> (дата обращения: 10.03.2026).
53. Top Game Development Frameworks & Engines for 2025 [Electronic resource] // Decipherzone. — 2025. — Access mode: <https://www.decipherzone.com/blog-detail/game-development-frameworks-engines> (дата обращения: 10.03.2026).
54. The Best Game Development Frameworks [Electronic resource] // GameFromScratch.com. — 2023. — Access mode: <https://gamefromscratch.com/the-best-game-development-frameworks/> (дата обращения: 10.03.2026).
55. Game Dev Experiences with SFML, Raylib, and SDL2 [Electronic resource] // GameDev.net Forums. — 2024. — Access mode: <https://www.gamedev.net/forums/topic/717635-game-dev-experiences-with-sfml-raylib-and-sdl2-share-your-insights/> (дата обращения: 10.03.2026).